

Please follow the initial setup instructions provided in the repository's GitHub README file before continuing with this document

Launching the application

After setting up virtual environment and installing dependencies and verifying the correct python version run command in the proj root directory (the folder in which you cloned the proj in eg- E:\EchoHands>

```
BASH
python -m src.app"
```

Since is the client-side part of the application, while its AI models are stored separately on a remote server. To work with these models, EchoHands caches them locally on your system.

When you launch EchoHands, the application first checks whether the required models are already available in its local cache.

If you have used EchoHands before and the models are still present in the cache, the application checks with the remote server whether those models are the latest available version. If they are up to date, EchoHands uses the cached models and proceeds to the main application interface.

If an older version of the models is present in the cache, EchoHands checks for the latest model package and retrieves the newer models before continuing to the main application.

If no models are found in the cache;

for example, when launching EchoHands for the first time, or when previously cached models have been cleared (yep, no-brainer—system cache is eventually cleared after a certain period) so EchoHands connects to the remote server, retrieves the latest model package, caches the models locally, and then starts the application.

This means the model files do not need to be permanently included inside the application itself. The local cache allows EchoHands to reuse the models when they are available, while still allowing newer model versions to be obtained whenever they are released.

Keeping Your Models Up to Date

EchoHands uses a Model Configuration File available in the official EchoHands GitHub repository to access the latest model release

Whenever a new model version is released, make sure you have the latest Model Configuration File in your EchoHands application directory. This allows EchoHands to access the newest available models when required

Whenever a new model version is released, update your local Model Configuration File before launching EchoHands.

► **The recommended way is to update your entire EchoHands application directory using Git and pull entire repo and replacing current directory in your system so that you'll get other updated files as well; Open a terminal inside your EchoHands application directory and run:**

1. Entering your EchoHands application directory

```
bash
cd <path-to-EchoHands>
```

For example:

```
bash
cd C:\Users\<YourName>\Desktop\EchoHands
```

2. Get the latest files from the official repository

```
bash
git pull
```

This will update your local EchoHands files, including the latest Model Configuration File.

3. Verify that the configuration file is present

You should see the following file inside your EchoHands application directory:

```
text
model_manifest.json
```

Note: You do not need to edit this file in any way. EchoHands reads it automatically when the application starts.

► **Updating Only the Model Configuration File**

If you do not want to update the rest of the application and only need the latest Model Configuration File, you can retrieve it directly.

Open a terminal inside your EchoHands application directory and run:

```
bash
curl.exe -L -o model_manifest.json
https://raw.githubusercontent.com/shivanshu43/Echohands/main/model_manifest.json
```

This replaces the existing Model Configuration File with the latest version available in the official EchoHands repository.

Recommended: Use git pull whenever possible, as a new model release may sometimes require changes to the application itself. The direct command should mainly be used when you specifically need to update only the Model Configuration File.

► **or You can simply get updated Configuration file directly from GitHub Repo ;**



Launching the application

Model Configuration file loaded

Configuration file validated

Check local model cache

not found /
Expired

if found

Download
required models

use cached
model

Download latest
model A/c to
configuration file

*It first establishes connection
with server & using HTTPS req
receives the model one at a time*

Preparing model download...

Connecting to the model server. The first connection may take a few seconds.

MODEL SETUP

No valid cached model package was found.
Downloading latest model package version v1.0.0
No valid cached model package was found.

Latest Model Version (from config file) v1.0.0

Using the info from configuration file detected the latest version available

Downloading random_forest.pkl
30.0 MB / 114.1 MB (26.3%) • 7.1 MB/s

Then after Starts downloading...

user might be
launching app for
the first time or
models expired

Verify the
Downloaded models

[✓] random_forest.pkl
[✓] label_encoder.pkl
[>] dynamic_lstm.keras
[] dynamic_label_encoder.npy

Both these models are downloaded & verified using their respective SHA-256 codes (to check if data was corrupted during transit)

Setting up new connection for the model; while next one waiting

verified verified
Connecting...
Waiting

OVERALL DOWNLOAD PROGRESS

99.8% 2 / 4 models verified

Load Both Static &
Dynamic models

Starts WebCam

Loading recognition engine...

Loading the verified recognition models. Please wait while the engine initializes.

MODEL SETUP

Retrieving models from cached package (v1.0.0)

Latest model version (from Configuration) v1.0.0

Shows the latest model version available after validating from server using Model configuration file

[✓] random_forest.pkl This is where you can see from where models are fetched Ready
[✓] label_encoder.pkl Ready
[✓] dynamic_lstm.keras Ready
[✓] dynamic_label_encoder.npy Ready

OVERALL DOWNLOAD PROGRESS

100.0% 4 / 4 models verified

Mode: NONE
Confidence: 63.0%
Status: Gesture Locked

HELLO TRISHA

Loading recognition engine...

Loading the verified recognition models. Please wait while the engine initializes.

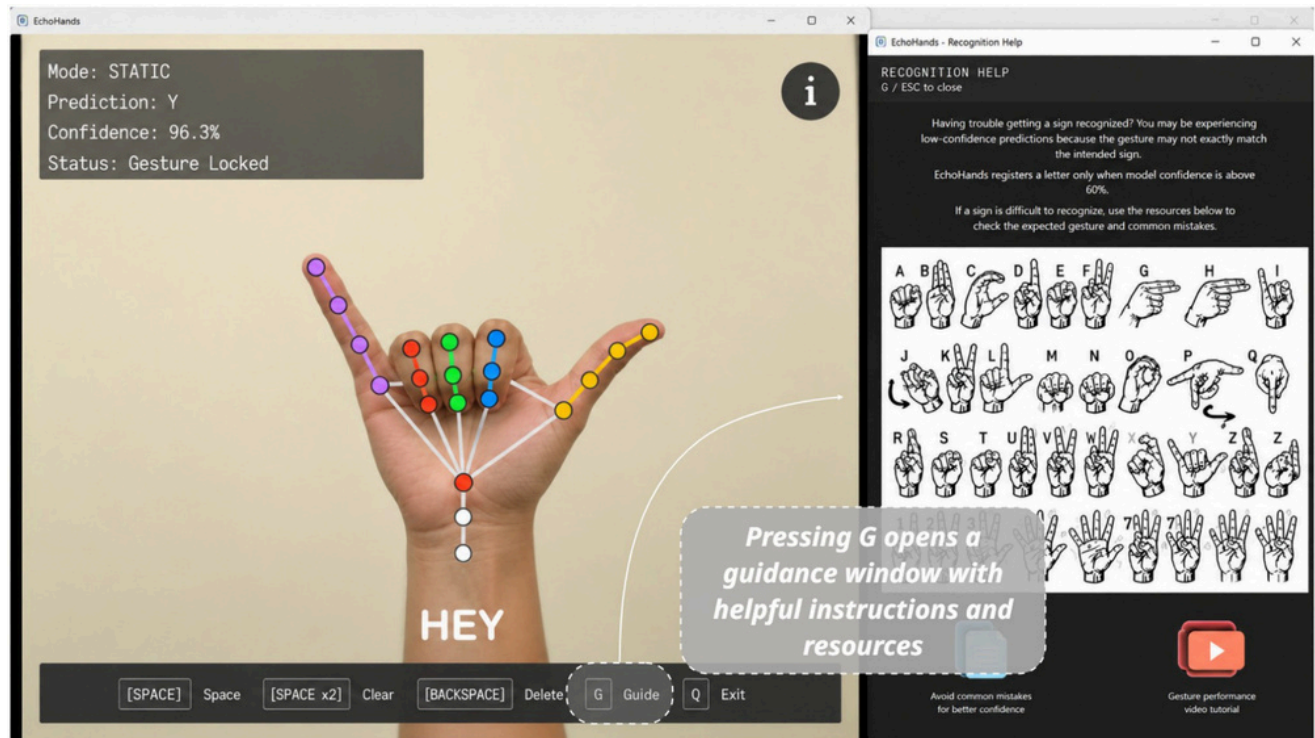
Starting camera...

Opening the webcam. Please allow camera access if Windows asks.



Directs user to
Main Interface

Understanding interface



Interface provides a real-time view of the recognition process. As the user interacts with the system through hand gestures and keyboard controls, the interface continuously updates to show what EchoHands is detecting, predicting, processing, and finally accepting

Mode

Mode: STATIC

The Mode indicator shows the recognition mode currently active in the application shows what kind of gesture EchoHands is currently processing.

- **NONE** — The model isn't processing a gesture at the moment. This is generally shown when no hand is detected on the screen.
- **STATIC** — The system switches to this mode when a static sign such as an alphabet or number is detected. These signs can be recognized from a single hand position.
- **DYNAMIC** — The system switches to this mode for gestures where movement matters, such as J and Z. This mode will also be used for future dynamic and compound gestures such as HELLO, GOOD, LOVE, and others planned for later development.

Prediction

Prediction: Y

Prediction: No Hand Detected

The Prediction shows what the recognition model is currently predicting from what it sees.

No Hand Detected — No hand is currently detected by the system.

A sign such as A, B, J, or Z — The model is currently predicting that sign.

The prediction is live, so it can change as you move your hand or transition between gestures. A prediction does not automatically mean that the sign has been accepted into the text.

Confidence

Confidence: 96.3%

The Confidence shows how strongly the model supports its current prediction.

EchoHands uses a fixed confidence threshold to decide whether a prediction is reliable enough to be considered for recognition. Predictions below this threshold are filtered out instead of being accepted as a gesture.

This helps reduce incorrect recognitions caused by unclear hand positions, movement, or uncertain predictions.

Status

The Recognition State shows what EchoHands is currently doing with the detected gesture.

When it shows:

Status: Gesture Locked

the system has finished processing the previous gesture and is ready for the next one.

The repeated-gesture behavior is managed by EchoHands' gesture locking mechanism

Once a gesture has been accepted, the locking mechanism prevents the same held gesture from being added repeatedly until the appropriate hand or gesture transition occurs.

For example, holding the sign A for several frames should result in:

text
A

rather than:

text
AAAA

Once the gesture is accepted and locked, EchoHands waits for the gesture to be released or transitioned before allowing another recognition.

Text Output

HEY

The Text area shows the signs that EchoHands has actually accepted as input and added to the current text. This part of the interface is managed by the Word Builder module, which keeps track of the accepted characters and builds them into words and text.

The important difference is:

Prediction : What the model currently predicts

Text : What EchoHands has accepted

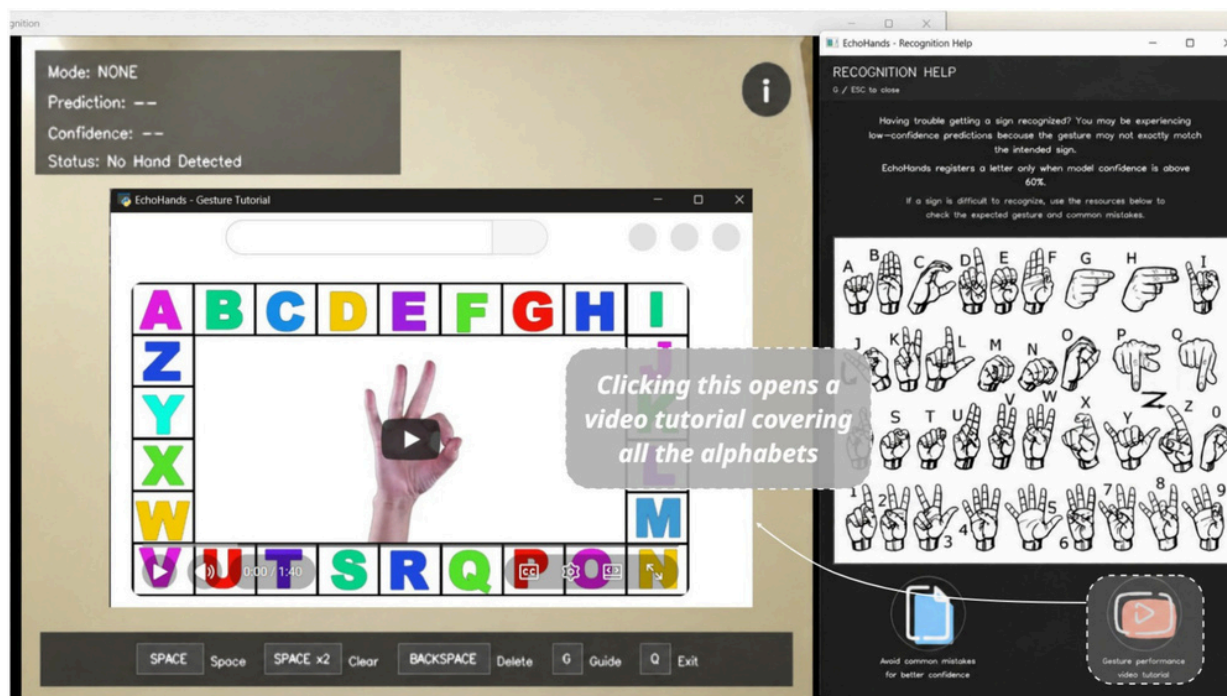
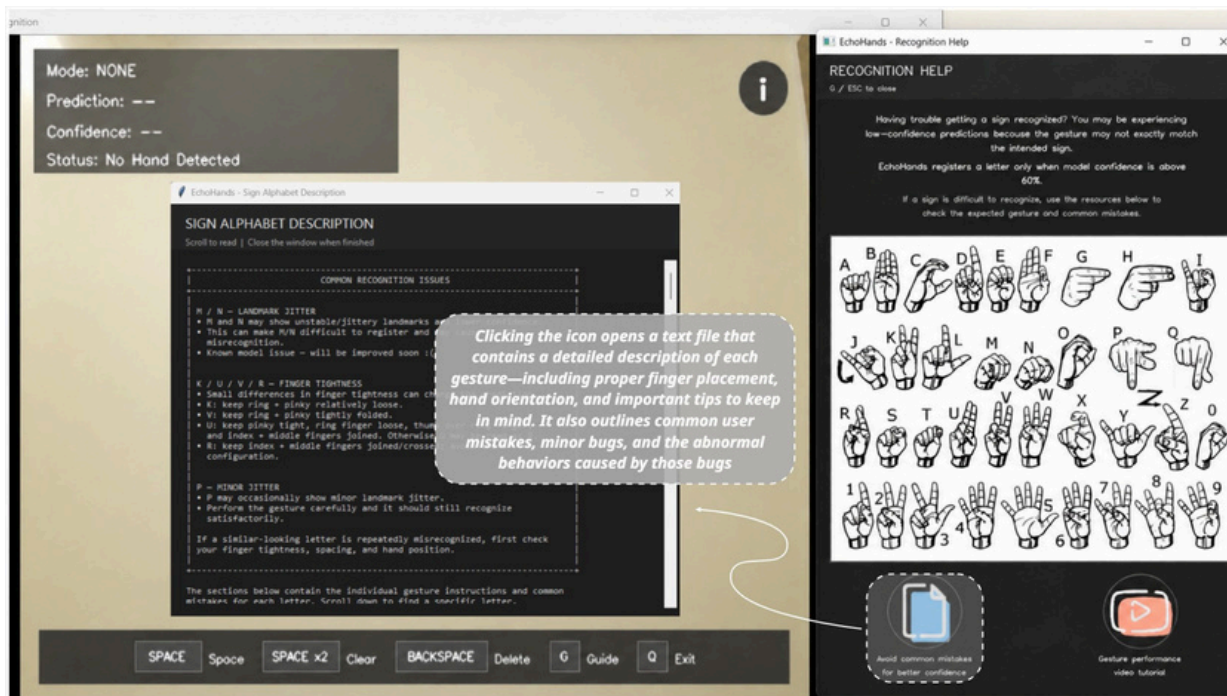
So, the prediction can change while you move between gestures, but the text is updated only when a gesture passes through the recognition-control process and is accepted

As the user performs accepted signs one by one, the text is built gradually.

Keyboard Controls

The keyboard controls are also handled through the Word Builder module and let you manage the generated text and the application.

Key	Behaviour
[SPACE]	Adds a space to the current text
[DOUBLE SPACE]	Clears the complete text output
[BACKSPACE]	Removes the last accepted character
[Q]	Closes the EchoHands application
[G]	Opens a guidance window with helpful instructions and resources



Supported Signs

EchoHands currently works with the trained signs included in the project.

The system handles two main categories of gestures:

Static Gestures

Static gestures are recognized primarily from the configuration or pose of the hand in a captured frame.

The current Alpha Build supports the trained static ASL alphabet and numeric signs available in the included model.

Dynamic Gestures

Some ASL signs involve motion and cannot be reliably recognized from a single frame.

EchoHands currently includes dedicated sequence-based recognition for: **J** and **Z**

These gestures are recognized by analyzing hand movement across multiple frames.

Supported Signs

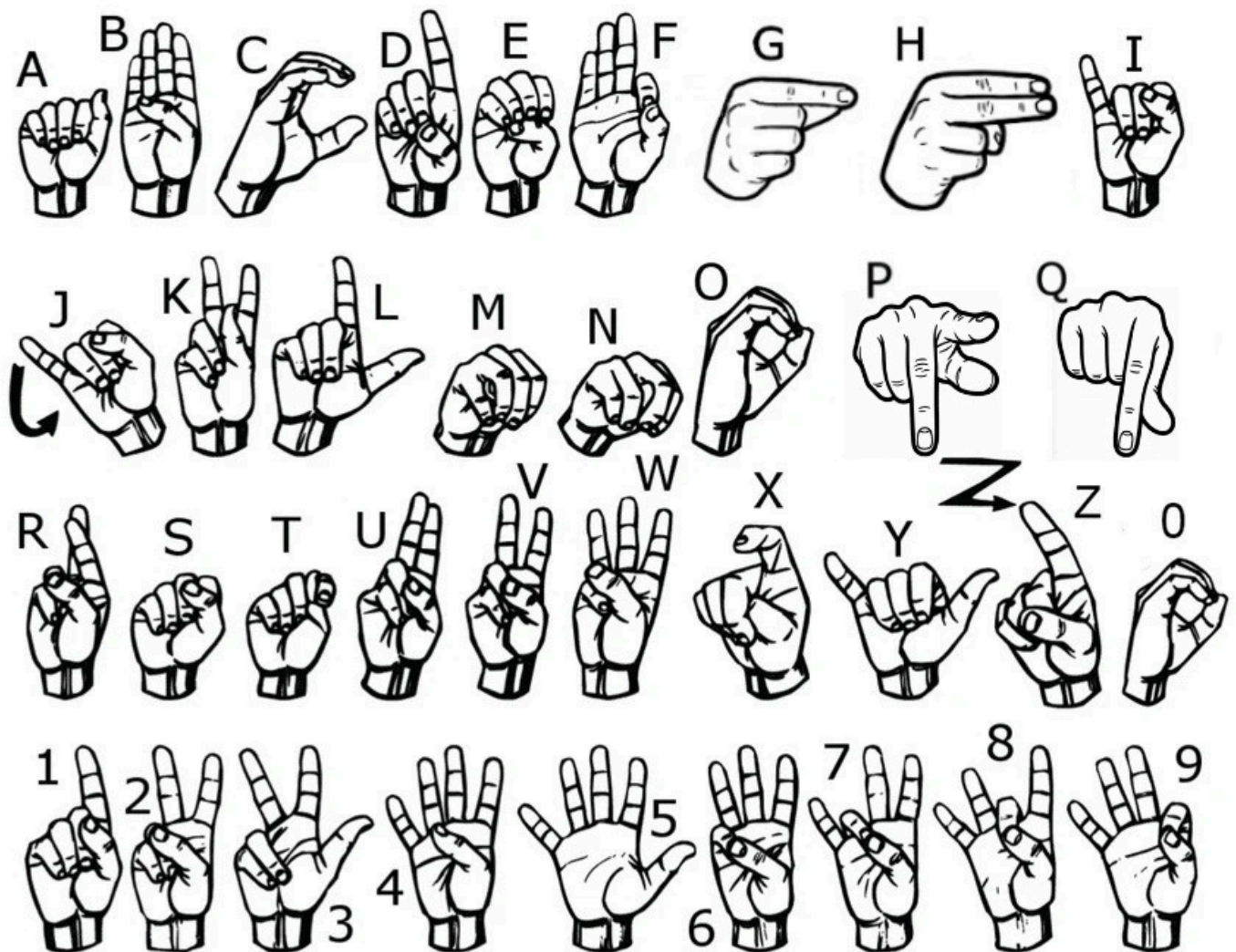


Fig. ASL alphabet and numeric signs supported by the EchoHands recognition system

How EchoHands Works

- WebcamCapture: Capture real-time video using OpenCV
- Hand Detection: Detect hands and extract 21 landmarks using MediaPipe
- Feature Processing: Process landmark data into model-ready features using NumPy
- Static Recognition: Recognize static signs using a scikit-learn Random Forest model
- Dynamic Recognition: Analyze multi-frame hand movements using a TensorFlow/Keras LSTM model
- Recognition Control: Check and filter model predictions before accepting them, ensuring a gesture is recognized only when it is reliable and preventing the same held gesture from being added repeatedly.
- Text Output: Build recognized signs into digital text through the application's text-building module

The main recognition pipeline is:

